SISTEMA DE ARRIENDO DE BICICLETAS Y SCOOTERS COMPARTIDOS

INTEGRANTES GRUPO 2:
BENJAMIN GOYCOLEA
FELIX MEDINA
IGNACIO TAPIA
CARLOS GONZALEZ
ESTEBAN ALIAGA

INTRODUCCIÓN

Plataforma tecnológica diseñada para resolver toda la operativa de arriendo de vehículos compartidos bajo una arquitectura moderna y desacoplada el cual permite una interraccion fluida y segura

El nucleo de este desarrollo reside en el motor de base de datos PostgreSQL, en el cual se implementaron objetos programables avanzados como procedimientos almacenados, triggers, vistas materializadas e indices estrategicos

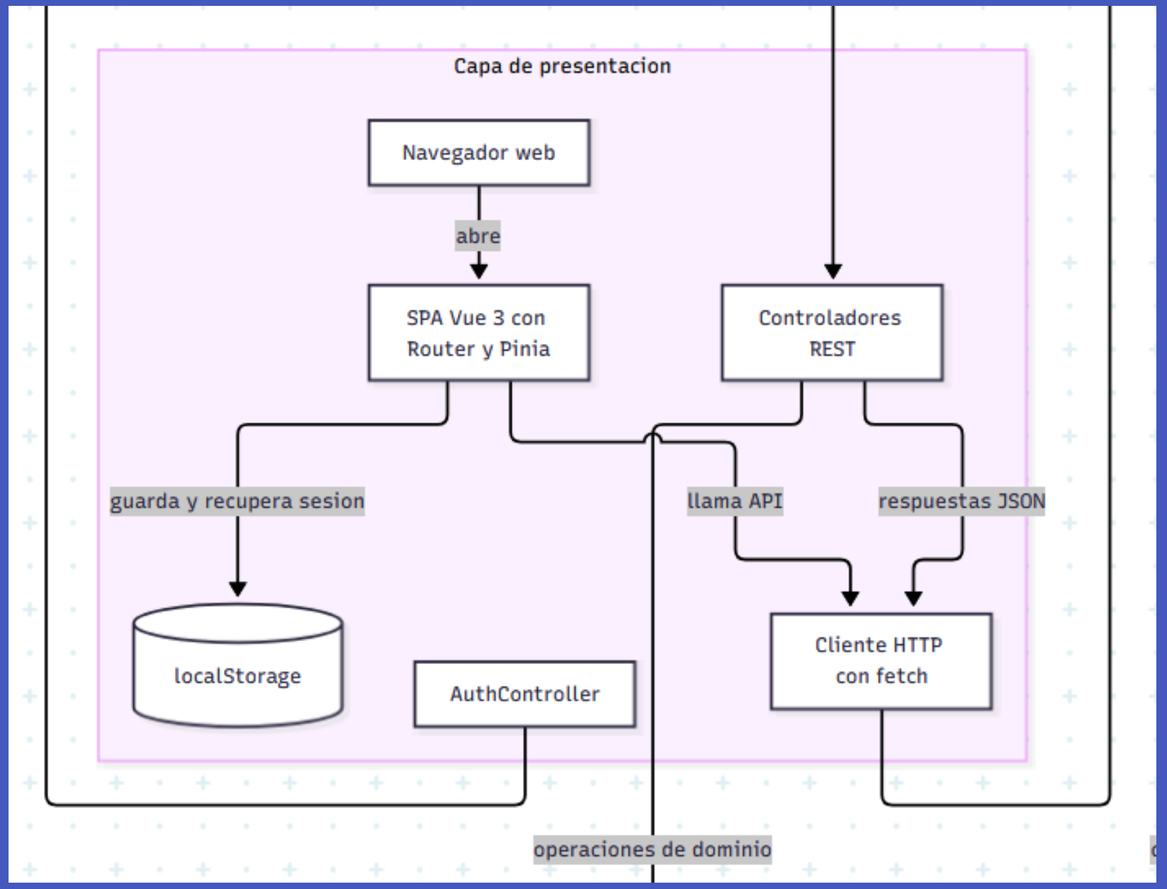
DESCRIPCIÓN AVANCE DE PROYECTO

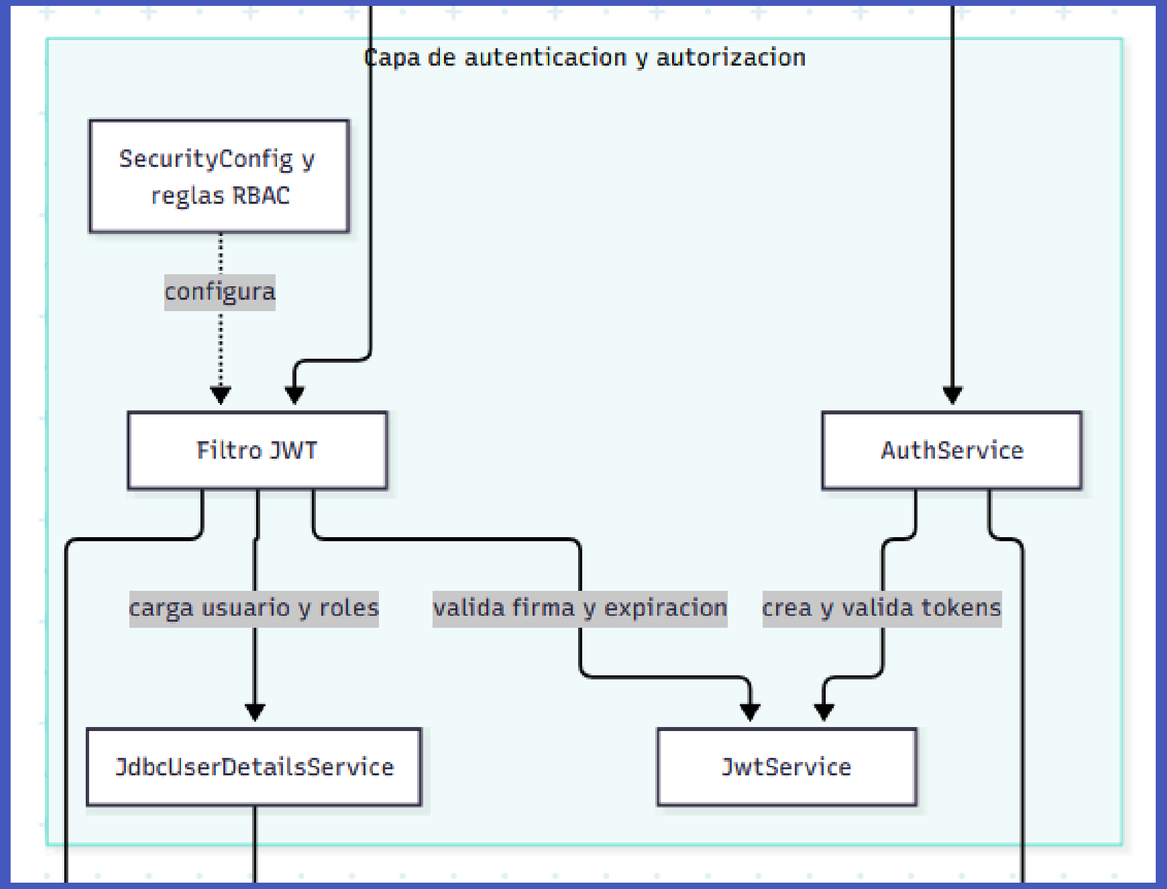


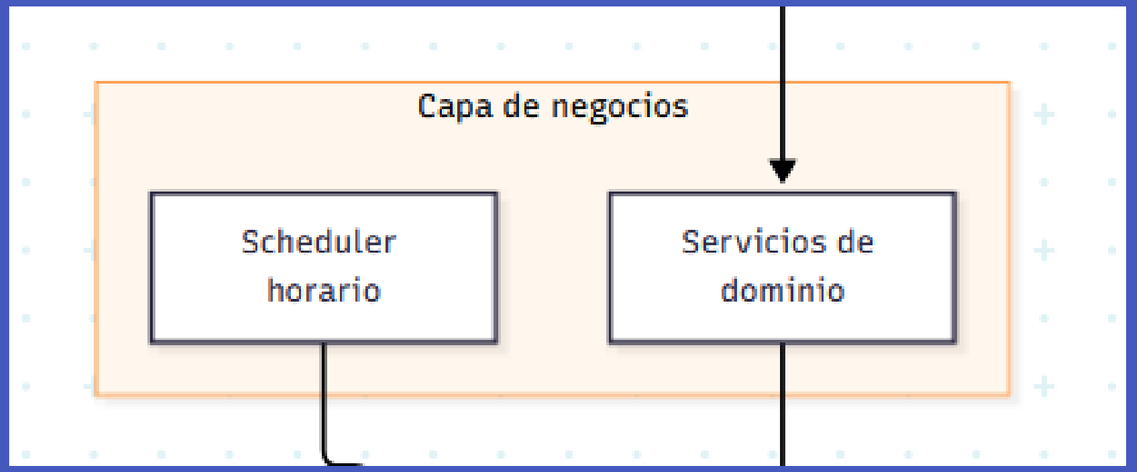
- El sistema cuenta con una arquitectura desacoplada totalmente funcional que integra un Backend en Java 21, un Frontend en Vue y una base de datos PostgreSQL.
- El entorno backend y de la base de datos se despliega de forma automatizada mediante Docker Compose, asegurando la ejecución de scripts SQL de inicialización.
- En la base de datos se logro implementar los procedimientos almacenado (finalizar viaje y el rebalanceo de flota), los triggers (validación de inicio de viaje y auditoria de flota) y las vistas materializadas (demanda por estación y franja horaria).
- Mientras que en el backend se integro el API REST (JdbcTemplate), los endpoints especiales que muestran la lista de disponibilidad por estación con promedio de batería y resumen mensual por usuario calculado en BD y una seguridad usando autenticacion por jwt con los roles definidos.
- Durante el desarrollo se hizo primero el backend, luego frontend y al final la base de datos, lo cual no deberia haber sido y por tanto pueden existir conflictos.

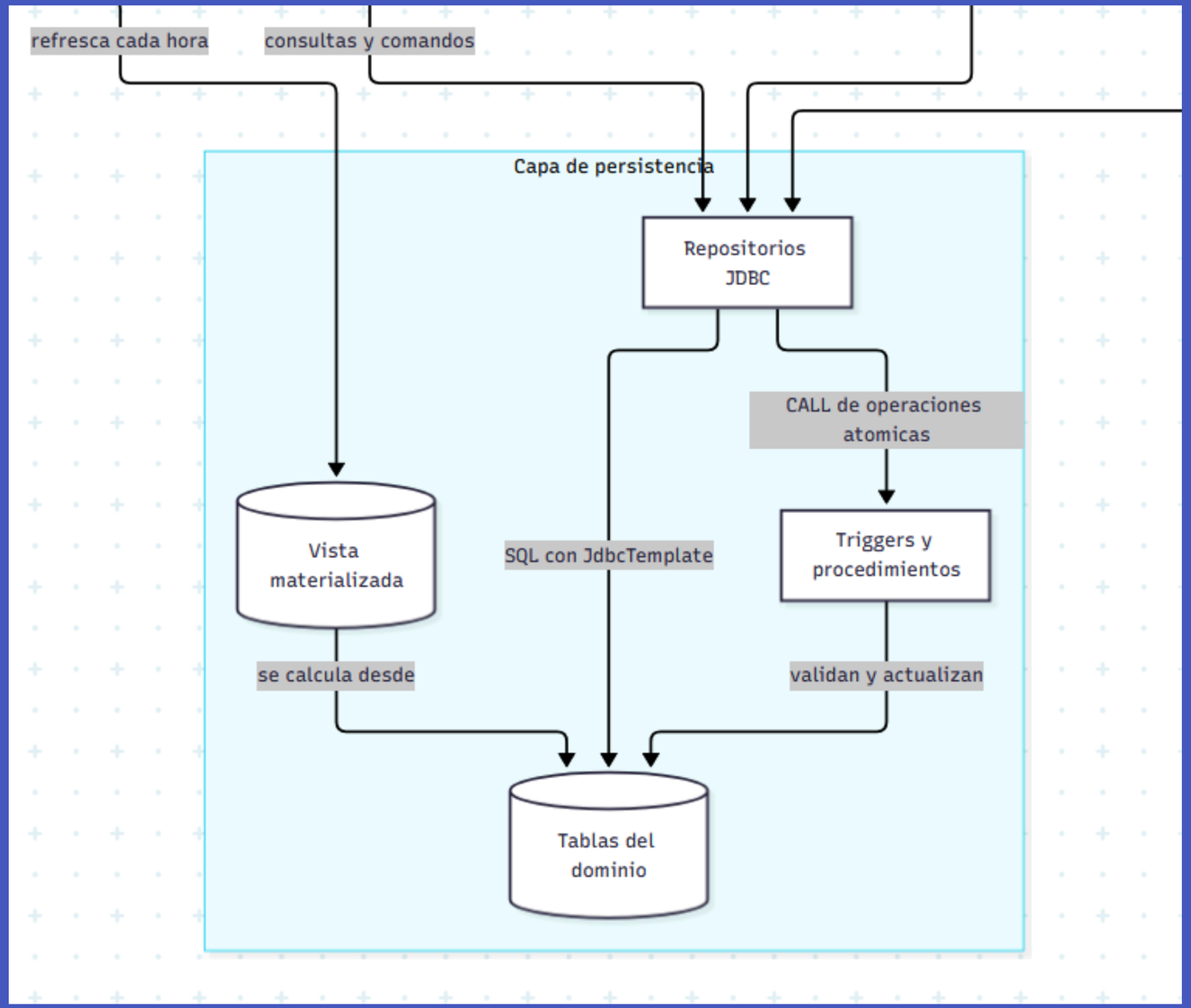
LOGO

1. DIAGRAMA DE ARQUITECTURA


Capa de presentacion
Navegador web
abre
SPA Vue 3 con Router y Pinia
Controladores REST
guarda y recupera sesion
llama API
respuestas JSON
localStorage
AuthController
Cliente HTTP con fetch
operaciones de dominio


Capa de autenticacion y autorizacion
SecurityConfig y reglas RBAC
configura
Filtro JWT
AuthService
carga usuario y roles
valida firma y expiracion
crea y valida tokens
JdbcUserDetailsService
JwtService


Capa de negocios
Scheduler horario
Servicios de dominio


refresca cada hora
consultas y comandos
Capa de persistencia
Repositorios JDBC
CALL de operaciones atomicas
Vista materializada
SQL con JdbcTemplate
Triggers y procedimientos
se calcula desde
validan y actualizan
Tablas del dominio

2. DEMOSTRACIÓN DE FUNCIONALIDADES

3. FRAGMENTOS DE CÓDIGO

FRAGMENTOS DE CÓDIGO: CONTROLLERS

```java
@RestController
@RequestMapping("/api/v1/usuarios")
public class UsuarioController {

    private final UsuarioService service;

    public UsuarioController(UsuarioService service) {
        this.service = service;
    }

    @GetMapping("/me")
    public UsuarioResponse me(Authentication authentication) {
        return UsuarioResponse.desde(
                service.findByUsername(authentication.getName())
        );
    }

    @GetMapping
    @PreAuthorize("hasRole('ADMIN')")
    public List<UsuarioResponse> findAll() {
        return service.findAll().stream()
                .map(UsuarioResponse::desde)
                .toList();
    }

    @GetMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public UsuarioResponse findById(@PathVariable long id) {
        return UsuarioResponse.desde(service.findById(id));
    }

    @GetMapping("/{id}/resumen-mensual")
    @PreAuthorize("hasRole('ADMIN')")
    public List<ResumenMensualResponse> obtenerResumenMensual(@PathVariable long id) {
        return service.obtenerResumenMensual(id);
    }

    @GetMapping("/me/resumen-mensual")
    public List<ResumenMensualResponse> obtenerMiResumenMensual(Authentication authentication) {
        long usuarioId = service.findByUsername(authentication.getName()).id();
        return service.obtenerResumenMensual(usuarioId);
    }
```

```java
@RestController
@RequestMapping("/api/v1/estaciones")
public class EstacionController {

    private final VehiculoService vehiculoService;
    private final EstacionService estacionService;

    public EstacionController(VehiculoService vehiculoService, EstacionService estacionService) {
        this.vehiculoService = vehiculoService;
        this.estacionService = estacionService;
    }

    @GetMapping
    @PreAuthorize("hasAnyRole('OPERADOR', 'ADMIN')")
    public List<Estacion> findAll() {
        return estacionService.findAll();
    }

    @GetMapping("/{id}")
    @PreAuthorize("hasAnyRole('OPERADOR', 'ADMIN')")
    public Estacion findById(@PathVariable long id) {
        return estacionService.findById(id);
    }

    @PostMapping
    @PreAuthorize("hasAnyRole('OPERADOR', 'ADMIN')")
    @ResponseStatus(HttpStatus.CREATED)
    public Estacion create(@Valid @RequestBody EstacionRequest request) {
        return estacionService.create(request);
    }

    @PutMapping("/{id}")
    @PreAuthorize("hasAnyRole('OPERADOR', 'ADMIN')")
    public Estacion update(@PathVariable long id, @Valid @RequestBody EstacionRequest request) {
        estacionService.update(id, request);
        return estacionService.findById(id);
    }

    @DeleteMapping("/{id}")
    @PreAuthorize("hasAnyRole('OPERADOR', 'ADMIN')")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void deactivate(@PathVariable long id) {
        estacionService.deactivate(id);
    }
```

FRAGMENTOS DE CÓDIGO: DTO

```java
package com.labtbd.dto;

import com.labtbd.entity.Rol;

public record AuthResponse(
        String token,
        String tokenType,
        String username,
        Rol rol
) {}
```

```java
package com.labtbd.dto;

import com.labtbd.entity.EstadoVehiculo;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;

public record CambiarEstadoRequest(
    @NotNull
    EstadoVehiculo estado,

    @NotBlank
    @Size(max=500)
    String motivo
) {}
```

```java
package com.labtbd.dto;

import com.labtbd.entity.TipoVehiculo;

import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotNull;

public record RebalanceoRequest(
    @NotNull
    Long estacionOrigenId,

    @NotNull
    Long estacionDestinoId,

    @NotNull
    TipoVehiculo tipoVehiculo,

    @Min(1)
    int cantidad
) {}
```

```java
package com.labtbd.dto;

import com.labtbd.entity.EstadoVehiculo;
import com.labtbd.entity.TipoVehiculo;
import com.labtbd.entity.Vehiculo;

public record VehiculoResponse(
    long id,
    String codigo,
    TipoVehiculo tipo,
    EstadoVehiculo estado,
    int bateria,
    Long estacionId
) {
    public static VehiculoResponse desde(Vehiculo v){
        return new VehiculoResponse(v.id(), v.codigo(), v.tipo(),
         v.estado(), v.bateria(), v.estacionId());
    }
}
```

FRAGMENTOS DE CÓDIGO: ENTITY

```java
package com.labtbd.entity;

public record Usuario(
    long id,
    String username,
    String nombre,
    String email,
    String passwordHash,
    Rol rol,
    boolean activo
) {}
```

```java
package com.labtbd.entity;

public enum Rol {
    USUARIO,
    OPERADOR,
    ADMIN
}
```

```java
package com.labtbd.entity;

import java.math.BigDecimal;
import java.time.OffsetDateTime;

public record Viaje(
        long id,
        long usuarioId,
        long vehiculoId,
        OffsetDateTime iniciadoEn,
        OffsetDateTime finalizadoEn,
        BigDecimal costo,
        Long estacionOrigenId,
        Long estacionDestinoId,
        Long tarifaId,
        BigDecimal precioPorMinutoAplicado,
        boolean cancelado
) {}
```

```java
package com.labtbd.entity;

import java.time.OffsetDateTime;
import java.util.UUID;

public record OrdenTraslado(
    long id,
    UUID solicitudId,
    long estacionOrigenId,
    long estacionDestinoId,
    TipoVehiculo tipoVehiculo,
    int cantidad,
    long operadorId,
    OffsetDateTime creadaEn
) {}
```

FRAGMENTOS DE CÓDIGO: REPOSITORY

```java
@Repository
public class OrdenTrasladoRepository {
    private final JdbcTemplate jdbc;

    private final RowMapper<OrdenTraslado> mapper = (rs, n) -> new OrdenTraslado(
            rs.getLong(columnLabel: "id"),
            rs.getObject(columnLabel: "solicitud_id", type: UUID.class),
            rs.getLong(columnLabel: "estacion_origen_id"),
            rs.getLong(columnLabel: "estacion_destino_id"),
            TipoVehiculo.valueOf(rs.getString(columnLabel: "tipo_vehiculo")),
            rs.getInt(columnLabel: "cantidad"),
            rs.getLong(columnLabel: "operador_id"),
            rs.getObject(columnLabel: "creada_en", type: java.time.OffsetDateTime.class));

    private static final String COLUMNS = """
            id, solicitud_id, estacion_origen_id, estacion_destino_id,
            tipo_vehiculo, cantidad, operador_id, creada_en
            """;

    public OrdenTrasladoRepository(JdbcTemplate jdbc) { this.jdbc = jdbc; }

    public List<OrdenTraslado> findAll() {
        return jdbc.query("SELECT " + COLUMNS + " FROM ordenes_traslado ORDER BY id DESC", mapper);
    }

    public Optional<OrdenTraslado> findById(long id) {
        return jdbc.query("SELECT " + COLUMNS + " FROM ordenes_traslado WHERE id = ?",
                mapper, id).stream().findFirst();
    }

    public Optional<OrdenTraslado> findBySolicitudId(UUID solicitudId) {
        return jdbc.query("SELECT " + COLUMNS + " FROM ordenes_traslado WHERE solicitud_id = ?",
                mapper, solicitudId).stream().findFirst();
    }

    public void ejecutarRebalanceo(UUID solicitudId, long origen, long destino,
                                   String tipo, int cantidad, long operadorId) {
        jdbc.update("CALL ejecutar_rebalanceo(?, ?, ?, ?, ?, ?)",
                solicitudId, origen, destino, tipo, cantidad, operadorId);
    }

    public Optional<Long> findOperadorId(String username) {
        return jdbc.queryForList("SELECT id FROM usuarios WHERE username = ?",
                Long.class, username).stream().findFirst();
    }
}
```

```java
@Repository
public class VehiculoRepository {
    private final JdbcTemplate jdbc;

    private final RowMapper<Vehiculo> mapper = (rs, n) -> new Vehiculo(
            rs.getLong(columnLabel: "id"), rs.getString(columnLabel: "codigo"),
            TipoVehiculo.valueOf(rs.getString(columnLabel: "tipo")),
            EstadoVehiculo.valueOf(rs.getString(columnLabel: "estado")),
            rs.getInt(columnLabel: "bateria"), rs.getObject(columnLabel: "estacion_id", type: Long.class));

    public VehiculoRepository(JdbcTemplate jdbc) { this.jdbc = jdbc; }

    public List<Vehiculo> findAll() {
        return jdbc.query("""
                SELECT id, codigo, tipo, estado, bateria, estacion_id
                FROM vehiculos ORDER BY id
                """, mapper);
    }

    public Optional<Vehiculo> findById(long id) {
        return jdbc.query("""
                SELECT id, codigo, tipo, estado, bateria, estacion_id
                FROM vehiculos WHERE id = ?
                """, mapper, id).stream().findFirst();
    }

    public Vehiculo create(VehiculoCreateRequestData d) {
        return jdbc.queryForObject("""
                INSERT INTO vehiculos (codigo, tipo, estado, bateria, estacion_id)
                VALUES (?, ?, 'DISPONIBLE', ?, ?)
                RETURNING id, codigo, tipo, estado, bateria, estacion_id
                """, mapper, d.codigo(), d.tipo().name(), d.bateria(), d.estacionId());
    }

    public int update(long id, String codigo, int bateria) {
        return jdbc.update("""
                UPDATE vehiculos SET codigo = ?, bateria = ?
                WHERE id = ? AND estado <> 'DADO_DE_BAJA'
                """, codigo, bateria, id);
    }
```

FRAGMENTOS DE CÓDIGO: SECURITY

```java
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;
    private final JdbcUserDetailsService userDetailsService;

    public JwtAuthenticationFilter(
            JwtService jwtService,
            JdbcUserDetailsService userDetailsService
    ) {
        this.jwtService = jwtService;
        this.userDetailsService = userDetailsService;
    }

    @Override
    protected void doFilterInternal(
            HttpServletRequest request,
            HttpServletResponse response,
            FilterChain filterChain
    ) throws ServletException, IOException {
        String authorization = request.getHeader("Authorization");
        if (authorization == null || !authorization.startsWith(prefix: "Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }
```

```java
        String token = authorization.substring(beginIndex: 7);
        try {
            String username = jwtService.extractUsername(token);
            if (SecurityContextHolder.getContext().getAuthentication() == null) {
                UserDetails userDetails = userDetailsService.loadUserByUsername(username);
                if (!jwtService.isTokenValid(token, userDetails)) {
                    response.sendError(HttpServletResponse.SC_UNAUTHORIZED);
                    return;
                }

                UsernamePasswordAuthenticationToken authentication =
                        new UsernamePasswordAuthenticationToken(
                                userDetails, null, userDetails.getAuthorities());
                authentication.setDetails(
                        new WebAuthenticationDetailsSource().buildDetails(request));
                SecurityContextHolder.getContext().setAuthentication(authentication);
            }
        } catch (JwtException | IllegalArgumentException | UsernameNotFoundException exception) {
            SecurityContextHolder.clearContext();
            response.sendError(HttpServletResponse.SC_UNAUTHORIZED);
            return;
        }

        filterChain.doFilter(request, response);
    }
}
```

FRAGMENTOS DE CÓDIGO: SECURITY

```java
@Service
public class JdbcUserDetailsService implements UserDetailsService {

    private final UsuarioRepository usuarioRepository;

    public JdbcUserDetailsService(UsuarioRepository usuarioRepository) {
        this.usuarioRepository = usuarioRepository;
    }

    @Override
    public UserDetails loadUserByUsername(String username) {
        Usuario usuario = usuarioRepository.findByUsername(username)
                .filter(Usuario::activo)
                .orElseThrow(() -> new UsernameNotFoundException("Credenciales inválidas"));

        return User.withUsername(usuario.username())
                .password(usuario.passwordHash())
                .authorities("ROLE_" + usuario.rol().name())
                .build();
    }
}
```

```java
@Service
public class JwtService {

    private final SecretKey signingKey;
    private final long expirationMinutes;

    public JwtService(
            @Value("${app.jwt.secret}") String secret,
            @Value("${app.jwt.expiration-minutes:60}") long expirationMinutes
    ) {
        if (expirationMinutes < 1) {
            throw new IllegalArgumentException(s: "JWT expiration must be at least one minute");
        }
        this.signingKey = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
        this.expirationMinutes = expirationMinutes;
    }

    public String generateToken(UserDetails userDetails) {
        Instant now = Instant.now();
        return Jwts.builder()
                .subject(userDetails.getUsername())
                .issuedAt(Date.from(now))
                .expiration(Date.from(now.plusSeconds(expirationMinutes * 60)))
                .signWith(signingKey)
                .compact();
    }

    public String extractUsername(String token) {
        String subject = extractClaims(token).getSubject();
        if (subject == null || subject.isBlank()) {
            throw new IllegalArgumentException(s: "JWT subject is missing");
        }
        return subject;
    }
```

FRAGMENTOS DE CÓDIGO: SERVICE

```java
@Service
public class AuditoriaService {
    private final AuditoriaRepository repo;
    private final VehiculoService vehiculos;

    public AuditoriaService(AuditoriaRepository repo, VehiculoService vehiculos) {
        this.repo = repo;
        this.vehiculos = vehiculos;
    }

    public List<AuditoriaEstadoVehiculo> findByVehiculoId(long vehiculoId) {
        vehiculos.findById(vehiculoId);
        return repo.findByVehiculoId(vehiculoId);
    }
}
```

```java
@Service
public class OrdenTrasladoService {
    private final OrdenTrasladoRepository repo;
    public OrdenTrasladoService(OrdenTrasladoRepository repo) { this.repo = repo; }

    public List<OrdenTraslado> findAll() { return repo.findAll(); }

    public OrdenTraslado findById(long id) {
        return repo.findById(id).orElseThrow(() -> new ResponseStatusException(
                HttpStatus.NOT_FOUND, "Orden de traslado no encontrada"));
    }

    @Transactional
    public OrdenTraslado rebalancear(RebalanceoRequest r, String username) {
        if (r.estacionOrigenId().equals(r.estacionDestinoId())) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
                    "La estación de origen debe ser distinta de la de destino");
        }
        long operadorId = repo.findOperadorId(username).orElseThrow(() ->
                new ResponseStatusException(HttpStatus.UNAUTHORIZED, "Operador no encontrado"));

        UUID solicitudId = UUID.randomUUID();
        repo.ejecutarRebalanceo(solicitudId, r.estacionOrigenId(), r.estacionDestinoId(),
                r.tipoVehiculo().name(), r.cantidad(), operadorId);

        return repo.findBySolicitudId(solicitudId).orElseThrow(() ->
                new ResponseStatusException(HttpStatus.INTERNAL_SERVER_ERROR,
                        "La orden no fue creada"));
    }
}
```

4. CONTRIBUCIONES

FRONTEND



BACKEND



4. CONCLUSIONES

CONCLUSION

En conclusión, el desarrollo del sistema demostró que el implementar objetos programables avanzados en PostgresSQL fue muy fundamental, debido a que garantizó la atomicidad e integridad en operaciones complejas de negocio evitando inconsistencias de concurrencia que la capa de aplicación por sí sola no siempre puede asegurar con el mismo nivel de robustez.

Asimismo el optar por el uso de Spring Boot utilizando JdbcTemplate y SQL explícito en lugar de un ORM pesado permitió un mejor control sobre el rendimiento de las consultas facilitando la optimización mediante índices estratégicos y vistas materializadas para analítica de alta complejidad con una latencia mínima.

¡GRACIAS POR

SU ATENCIÓN!